Reporte Técnico de Actividades Práctico Experimentales Nro 006

1. Datos de identificación del estudiante y la práctica

Campo	Información
Tipo	Informe individual
Asignatura	Simulación
Ciclo	5
Unidad	U2: Generación de números pseudoaleatorios y generación de variables aleatorias
Resultado de aprendizaje de la unidad	R2. Interpreta los resultados de una Simulación, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	06
Título de la Práctica	Validación estadística de números pseudoaleatorios mediante Chi-cuadrado y Kolmogorov-Smirnov
Nombre del Docente	José O. Guamán Q.
Nombre del Estudiante	Yandri A. Piscocama J.
Fecha	05 de noviembre de 2026 06 de noviembre de 2026
Horario	Jueves 10h30 a 11h30 Viernes 07h30 a 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3

2. Objetivos de la práctica

Objetivo general

Evaluar la calidad estadística de una secuencia de números pseudoaleatorios mediante pruebas de bondad de ajuste, utilizando los métodos Chi-cuadrado y Kolmogorov-Smirnov para determinar si los valores generados presentan un comportamiento compatible con una distribución uniforme en [0,1].

Objetivos específicos

• Generar secuencias mediante el método congruencial lineal (LCG).
• Aplicar Chi-cuadrado para evaluar la uniformidad de las frecuencias observadas.
• Aplicar Kolmogorov-Smirnov para comparar la distribución empírica con la uniforme teórica.
• Interpretar estadísticos y valores p.
• Comparar diferentes configuraciones del generador.

3. Materiales, reactivos, equipos y herramientas

• Computador y Visual Studio Code.

• Python 3, NumPy, Pandas, Matplotlib, SciPy y entorno virtual (.venv).
• Java (JDK) y PowerShell.
• Archivos CSV, gráficas y pruebas automatizadas.

4. Procedimiento y metodología ejecutada

4.1. Fundamento teórico

Números pseudoaleatorios. Son valores producidos por procedimientos deterministas que permiten generar secuencias reproducibles y estudiar propiedades de interés en simulación. La uniformidad marginal consiste en que las proporciones de valores en subintervalos de igual longitud sean aproximadamente iguales.

Generador congruencial lineal (LCG). Se obtiene un nuevo estado por la recurrencia $X(n+1) = (a \cdot X(n) + c) \bmod m$, donde a es el multiplicador, c el incremento, m el módulo y X(0) la semilla. Se normaliza mediante $U(n+1) = X(n+1)/m$. El período depende de la configuración del generador.

Prueba Chi-cuadrado. Contrasta las frecuencias observadas O_i con las esperadas E_i mediante $\chi^2 = \Sigma[(O_i - E_i)^2/E_i]$. En esta actividad n = 10 000, k = 10, E_i = 1 000 y grados de libertad = 9 [1].

Prueba Kolmogorov-Smirnov. Compara la función de distribución acumulada empírica con la teórica. Para una uniforme en [0,1], F(x)=x. Con la muestra ordenada, se calcula $D^+ = \text{máx}[i/n - U(i)]$ y $D^- = \text{máx}[U(i) - (i-1)/n]$; $D = \text{máx}(D^+, D^-)$ [2].

Hipótesis y decisión. H_0: la distribución es uniforme U(0,1); H_1: no es uniforme. Se usa $\alpha = 0{,}05$: si $p < 0{,}05$, se rechaza H_0; de lo contrario, no se rechaza. No rechazar H_0 no demuestra que sea verdadera [3].

Precaución estadística. Los valores de un LCG son discretos y deterministas. Las calibraciones habituales de valores p suponen condiciones, en particular independencia, que estas secuencias no necesariamente satisfacen. Por ello las decisiones se interpretan como resultados descriptivos de las pruebas implementadas, no como certificación de aleatoriedad [2], [3].

Estimación complementaria de π. Para cubrir la tabla de resultados esperados, se empleó Monte Carlo con pares de valores uniformes (x, y) del LCG. Se cuenta cuántos de N puntos cumplen $x^2 + y^2 \leq 1$ y se estima $\pi = 4 \cdot (\text{puntos interiores}/N)$; el error absoluto es $|\pi \text{ estimado} - \pi|$. Este experimento es complementario: la guía incluye la tabla, pero no detalla el algoritmo.

4.2. Procedimiento realizado

1. Se revisó la guía APE 6 y se organizaron el código fuente, los datos y las evidencias.
2. Se implementó el LCG en Python y Java, con parámetros configurables y exportación de índice, estado entero y valor normalizado en CSV.
3. La configuración A utilizó semilla 42, a = 1664525, c = 1013904223, m = 4294967296 y n = 10000.
4. Se ejecutaron ambos generadores y se contrastaron sus CSV completos mediante SHA-256.
5. Se aplicó Chi-cuadrado con diez intervalos y KS contra U(0,1) utilizando $\alpha = 0{,}05$. Se conservaron tablas y gráficas generadas desde los datos.
6. Para contraste se configuró B con semilla 42, a = 1, c = 1, m = 256 y n = 10000, manteniendo el tamaño de muestra y el criterio estadístico.
7. Se compararon los resultados A/B y se ejecutaron pruebas automatizadas para comprobar reproducibilidad, coincidencia y funcionamiento.

8. Como ampliación de los resultados esperados, se estimó π mediante Monte Carlo en Python y Java para N = 1 000, 10 000, 100 000 y 1 000 000 puntos. Ambos programas utilizaron la misma configuración A y dos números LCG por punto; se verificaron conteos iguales entre lenguajes. Los tiempos de cálculo excluyen impresión y escritura, y corresponden a las medianas de cinco ejecuciones después del calentamiento.

5. Resultados obtenidos

Se contrastaron dos configuraciones del LCG con 10 000 valores cada una y un nivel de significancia de $\alpha = 0,05$. Los resultados siguientes proceden de los archivos CSV y las figuras originales de la práctica.

5.1. Generación y verificación entre Python y Java

Para cada configuración, las implementaciones de Python y Java generaron archivos CSV idénticos, lo que se comprobó mediante SHA-256. Los resultados completos se conservan en la carpeta de datos del proyecto; la comparación de hashes proporciona evidencia de igualdad byte por byte.

5.2. Prueba Chi-cuadrado: configuración A

Tabla 1. Frecuencias observadas y esperadas — configuración A.

Intervalo	Observada	Esperada
[0.0, 0.1)	1005	1 000
[0.1, 0.2)	1011	1 000
[0.2, 0.3)	964	1 000
[0.3, 0.4)	963	1 000
[0.4, 0.5)	1009	1 000
[0.5, 0.6)	940	1 000
[0.6, 0.7)	1039	1 000
[0.7, 0.8)	1013	1 000
[0.8, 0.9)	1030	1 000
[0.9, 1.0]	1026	1 000
Total	10 000	10 000

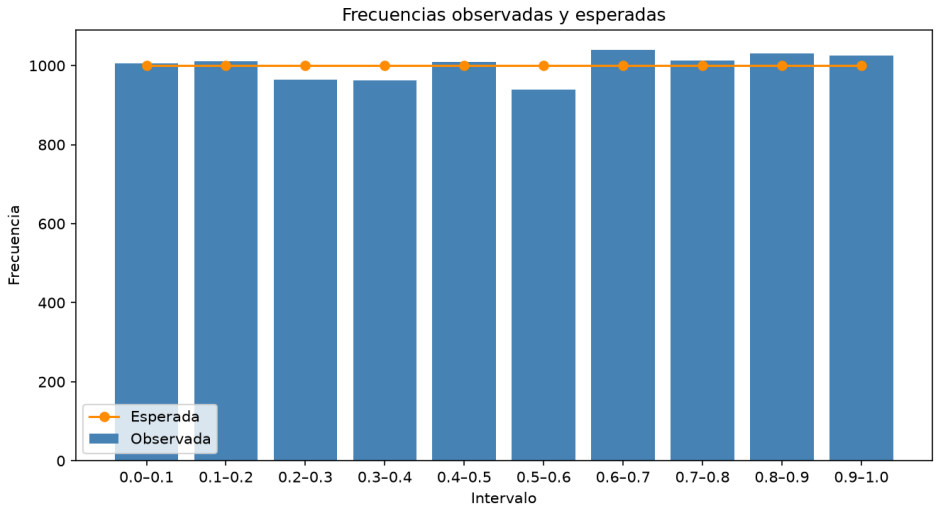


Figura 1. Frecuencias observadas y esperadas de la configuración A.

Las frecuencias oscilaron entre 940 y 1039. El resultado fue $\chi^2 = 9,758$ con 9 grados de libertad y $p = 0,3704295$. Puesto que $p > 0,05$, no se rechazó H_0 para la prueba de frecuencias.

5.3. Prueba Kolmogorov-Smirnov: configuración A

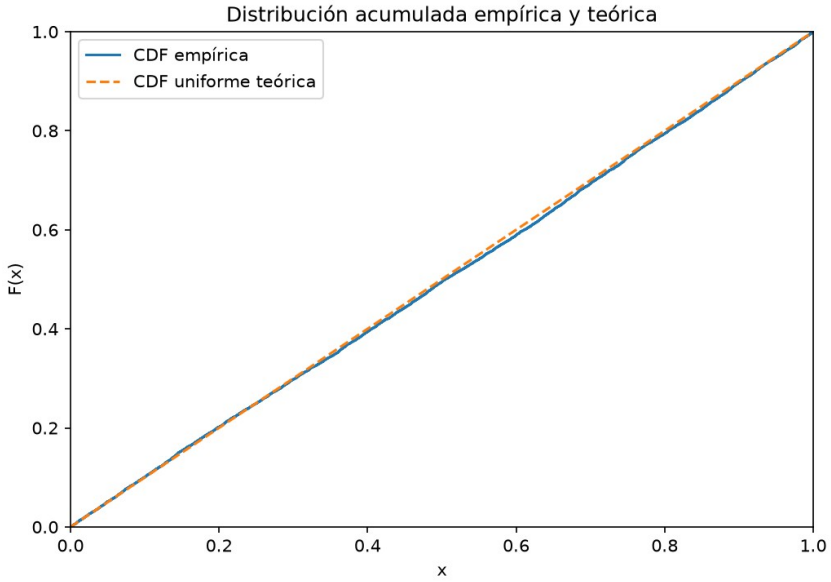


Figura 2. Distribución acumulada empírica y teórica — configuración A.

Se calcularon $D^+ = 0{,}0047128$, $D^- = 0{,}0125061$ y $D = 0{,}0125061$, con $p = 0{,}0868668$. Las curvas son visualmente cercanas. Como $p > 0{,}05$, tampoco se rechazó H_0, con las reservas relativas a los supuestos de la prueba.

5.4. Comparación entre configuraciones A y B

Tabla 2. Parámetros y resultados estadísticos de ambas configuraciones.

Indicador	**A**	**B**
Semilla	42	42
Multiplicador (a)	1664525	1
Incremento (c)	1013904223	1
Módulo (m)	4294967296	256
N	10000	10000
χ^2	9,758	3,708
p (χ^2)	0,3704295	0,9295594
D (KS)	0,0125061	0,0051375
p (KS)	0,0868668	0,9532291
Decisión en ambas	No rechazar H_0	No rechazar H_0

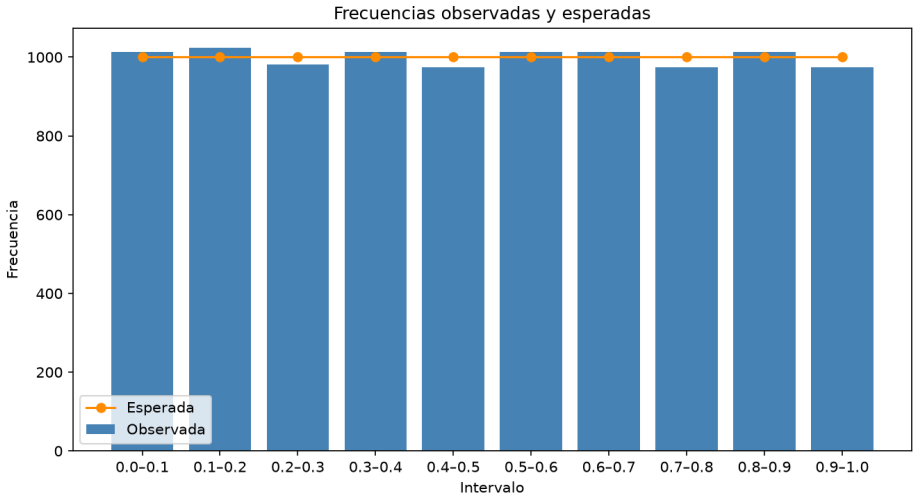


Figura 3. Frecuencias observadas y esperadas de la configuración B.

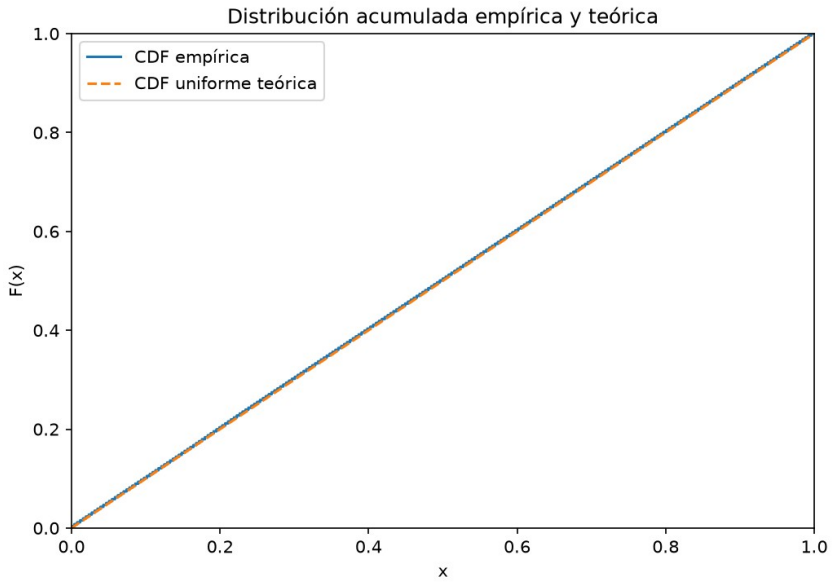


Figura 4. Distribución acumulada empírica y teórica — configuración B.

La configuración B recorre los 256 estados consecutivos antes de repetirse. En 10 000 observaciones contiene numerosas repeticiones de ese ciclo. Aunque en ambas pruebas sus valores p fueron mayores que los de A, ello no indica que sea un generador superior: el ajuste marginal a uniformidad no mide por sí mismo independencia, imprevisibilidad ni período [3].

5.5. Estimación de π y comparación de tiempos

La Tabla 3 presenta los resultados experimentales de Python y Java utilizando los mismos parámetros del LCG. El valor de π estimado y su error absoluto son comunes a ambos programas porque los conteos de puntos interiores coincidieron. Los tiempos están expresados en segundos y reflejan mediciones realizadas en el equipo de prueba.

Tabla 3. Estimación de π, error absoluto y tiempos de ejecución en Python y Java.

N	π estimado	Error absoluto	Python (s)	Java (s)
1 000	3,092000	0,049593	0,0005467	0,0001122
10 000	3,119200	0,022393	0,0041993	0,0004852
100 000	3,149960	0,008367	0,0414920	0,0046775
1 000 000	3,142712	0,001119	0,4489828	0,0467958

Interpretación. En estas ejecuciones, el error absoluto disminuyó de 0,049593 a 0,001119 al incrementar N de 1 000 a 1 000 000. Python y Java obtuvieron las mismas estimaciones; Java registró menores tiempos de cálculo en los cuatro tamaños medidos. Esta tendencia del error no está garantizada para cualquier muestra, y los tiempos dependen del equipo, la implementación y las condiciones de medición.

6. Preguntas de control

1. ¿Qué es la hipótesis nula?

Es la afirmación que se usa como referencia en un contraste estadístico. Aquí H_0 plantea que los datos siguen una distribución uniforme U(0,1) [1], [2].

2. ¿Qué representa el valor p?

Bajo H_0 y los supuestos de la prueba, es la probabilidad de obtener un estadístico al menos tan extremo como el observado. No es la probabilidad de que H_0 sea cierta [3].

3. ¿Qué significa rechazar H_0?

Significa que los resultados son incompatibles con H_0 bajo el nivel de significancia establecido. Con $\alpha = 0,05$, se rechaza cuando $p < 0,05$; en estas ejecuciones no se rechazó en ninguna de las dos configuraciones [3].

4. ¿Qué diferencia existe entre Chi-cuadrado y KS?

Chi-cuadrado compara frecuencias observadas y esperadas en intervalos; KS compara las distribuciones acumuladas y utiliza su diferencia máxima [1], [2].

5. ¿Por qué una prueba estadística no demuestra que un generador sea verdaderamente aleatorio?

Porque examina propiedades específicas de una muestra finita. El generador B, aunque no rechaza H_0, repite su secuencia cada 256 estados; las pruebas de uniformidad no garantizan independencia ni imprevisibilidad [3].

7. Conclusiones

1. La implementación del generador LCG en Python y Java permitió obtener secuencias de 10 000 números pseudoaleatorios con resultados idénticos al utilizar los mismos parámetros. Esto permitió comprobar el correcto funcionamiento de ambos programas y la importancia de la reproducibilidad en los experimentos de simulación.
2. Las pruebas Chi-cuadrado y Kolmogorov-Smirnov permitieron evaluar la uniformidad de los números generados. En ambas configuraciones no se rechazó la hipótesis nula con un nivel de significancia de 0,05. Sin embargo, se comprobó que obtener resultados favorables no garantiza que un generador sea verdaderamente aleatorio, ya que también deben considerarse características como su período y la dependencia entre valores.

8. Referencias bibliográficas

[1] NIST/SEMATECH, “Chi-Square Goodness of Fit Test,” Dataplot Reference Manual, National Institute of Standards and Technology. [En línea]. Disponible en: https://itl.nist.gov/div898/software/dataplot/refman1/auxillar/chsqgood.htm

[2] NIST/SEMATECH, “Kolmogorov-Smirnov Goodness-of-Fit Test,” e-Handbook of Statistical Methods, National Institute of Standards and Technology. [En línea]. Disponible en: https://www.itl.nist.gov/div898/handbook/eda/section3/eda35g.htm

[3] R. L. Wasserstein and N. A. Lazar, “The ASA Statement on p-Values: Context, Process, and Purpose,” The American Statistician, vol. 70, no. 2, pp. 129–133, 2016, doi: 10.1080/00031305.2016.1154108.

9. Anexos

9.1. Repositorio del proyecto

Simulación Portafolio en GitHub

9.2. Declaración de uso de inteligencia artificial

Se utilizó Codex (OpenAI) como apoyo para generar el código del generador congruencial lineal (LCG) en Python y Java, la estimación complementaria de π mediante Monte Carlo, así como las pruebas estadísticas Chi-cuadrado y Kolmogorov-Smirnov en Python. Para orientar este desarrollo, el estudiante analizó la guía y especificó los parámetros, los objetivos y los criterios de validación de la práctica. Posteriormente, se revisó el funcionamiento de los programas, se compararon las secuencias y se interpretaron las tablas y gráficas. También se utilizó ChatGPT como apoyo para adaptar el formato del informe y revisar la redacción y las referencias.

9.3. Resúmenes de prompts utilizados

Los siguientes textos resumen las instrucciones trabajadas y no constituyen transcripciones literales de los mensajes originales.

1. Generación del código en Python y Java

Genera un proyecto en Python y Java que implemente un generador congruencial lineal configurable y produzca 10 000 números pseudoaleatorios con los mismos parámetros. Verifica que ambas implementaciones generen la misma secuencia y almacena los resultados en CSV. Incorpora las pruebas Chi-cuadrado y Kolmogorov-Smirnov en Python, con $\alpha = 0{,}05$, y organiza el proyecto para comprobar su funcionamiento.

2. Visualización y análisis de resultados

Genera con Matplotlib una gráfica de las frecuencias observadas y esperadas y otra de las distribuciones acumuladas empírica y teórica, con títulos, ejes y leyendas. Compara las configuraciones A y B mediante sus estadísticos, valores p y período, e interpreta los resultados sin afirmar que no rechazar la hipótesis nula demuestre verdadera aleatoriedad.